UNIT II

Software Requirements Engineering and Analysis

Modeling:

Requirements Engineering :

- The requirements for a system are the descriptions of what the system should do—the services that it provides and the constraints on its operation. These requirements reflect the needs of customers for a system that serves a certain purpose such as controlling a device, placing an order, or finding information. The process of finding out, analyzing, documenting and checking these services and constraints is called Requirements Engineering (RE).
- Requirements engineering (RE) refers to the process of defining, documenting, and maintaining requirements in the engineering design process. Requirement engineering provides the appropriate mechanism to understand what the customer desires, analyzing the need, and assessing feasibility, negotiating a reasonable solution, specifying the solution clearly, validating the specifications and managing the requirements as they are transformed into a working system. Thus, requirement engineering is the disciplined application of proven principles, methods, tools, and notation to describe a proposed system's intended behavior and its associated constraints.

Requirement Engineering Process:

- it is a four-step process, which includes -

- The requirements engineering process has many potential pitfalls. Strengthening your RE process enables you to avoid many common challenges. The process serves as a compass, guiding you in determining the exact deliverables and doing so with greater accuracy. All important parties are on the same page about what steps need to be taken to achieve the desired outcome. Strengthen your RE process by considering the following:

- **Elicit requirements**:
  Elicitation is about becoming familiar with all the important details involved with the project. The customer will provide details about their needs and furnish critical background information. You will study those details and also become familiar with similar types of software solutions. This step provides important context for development.
- **Requirements specification:**
  During the specification phase, you gather functional and nonfunctional project requirements. A variety of tools are used during this stage, including data flow diagrams, to add more clarity to the project goals.

- **Verification and validation:**
  Verification ensures the software is built to the customer's requirements. In contrast, validation ensures the software is implementing the right functions. If the requirements don't go through the validation stage, there is the potential for time-consuming and expensive reworks.
- **Requirements management:**
  RM is an ongoing process that runs in parallel to the other three processes just described. In RM, you're matching all the relevant processes to their requirements. You will analyze, document, and prioritize the requirements – and communicate with relevant stakeholders. Any requirements that need modification are handled in an efficient and systematic manner.

- As you implement the different components of RE, it also helps you get a sense of what should be excluded from requirements. Understanding this will help you focus more accurately on developing requirements and better meeting client expectations.

Establishing the Groundwork:

- In an ideal setting, stakeholders and software engineers work together on the same team. In such cases, requirements engineering is simply a matter of conducting meaningful conversations with colleagues who are well-known members of the team. But reality is often quite different.
- Customer(s) or end users may be located in a different city or country, may have only a vague idea of what is required, may have conflicting opinions about the system to be built, may have limited technical knowledge, and may have limited time to interact with the requirements engineer. None of these things are desirable, but all are common, and you are often forced to work within the constraints imposed by this situation.

Identifying Stakeholders:

- Sommerville and Sawyer define a stakeholder as "anyone who benefits in a direct or indirect way from the system which is being developed." I have already identified the usual suspects: business operations managers, product managers, marketing people, internal and external customers, end users, consultants, product engineers, software engineers, support and maintenance engineers, and others. Each stakeholder has a different view of the system, achieves different benefits when the system is successfully developed, and is open to different risks if the development effort should fail.
- At inception, you should create a list of people who will contribute input as requirements are elicited. The initial list will grow as stakeholders are contacted because every stakeholder will be asked: "Whom else do you think I should talk to?"

Recognizing Multiple View Points:

- Because many different stakeholders exist, the requirements of the system will be explored from many different points of view. For example, the marketing group is interested in functions and features that will excite the potential market, making the new system easy to sell. Business managers are interested in a feature set that can be built within budget and that will be ready to meet defined market windows. End users may want features that are familiar to them and that are easy to learn and use. Software engineers may be concerned with functions that are invisible to nontechnical stakeholders but that enable an infrastructure that supports more marketable functions and features. Support engineers may focus on the maintainability of the software.
- Each of these constituencies (and others) will contribute information to the requirements engineering process. As information from multiple viewpoints is collected, emerging requirements may be inconsistent or may conflict with one another. You should categorize all stakeholder information (including inconsistent and conflicting requirements) in a way that will allow decision makers to choose an internally consistent set of requirements for the system.

Working toward Collaboration:

- If five stakeholders are involved in a software project, you may have five (or more) different opinions about the proper set of requirements. customers (and other stakeholders) must collaborate among themselves (avoiding petty turf battles) and with software engineering practitioners if a successful system is to result. But how is this collaboration accomplished?
- The job of a requirements engineer is to identify areas of commonality (i.e., requirements on which all stakeholders agree) and areas of conflict or inconsistency (i.e., requirements that are desired by one stakeholder but conflict with the needs of another stakeholder). It is, of course, the latter category that presents a challenge.
- Collaboration does not necessarily mean that requirements are defined by committee. In many cases, stakeholders collaborate by providing their view of requirements, but a strong “project champion” (e.g., a business manager or a senior technologist) may make the final decision about which requirements make the cut.

Asking the first Questions:

- Questions asked at the inception of the project should be “context free” . The first set of context-free questions focuses on the customer and other stakeholders, the overall project goals and benefits.
- For example, you might ask:
  - Who is behind the request for this work?
  - Who will use the solution?
  - What will be the economic benefit of a successful solution?
  - Is there another source for the solution that you need?

- These questions help to identify all stakeholders who will have interest in the software to be built. In addition, the questions identify the measurable benefit of a successful implementation and possible alternatives to custom software development.
- The next set of questions enables you to gain a better understanding of the problem and allows the customer to voice his or her perceptions about a solution:

    • How would you characterize "good" output that would be generated by a successful solution?

    • What problem(s) will this solution address?

    • Can you show me (or describe) the business environment in which the solution will be used?

    • Will special performance issues or constraints affect the way the solution is approached?

- The final set of questions focuses on the effectiveness of the communication activity itself. Gause and Weinberg call these "meta-questions" and propose the following (abbreviated) list:

    • Are you the right person to answer these questions?

    Are your answers "official"?

    • Are my questions relevant to the problem that you have?

    • Am I asking too many questions?

    • Can anyone else provide additional information?

    • Should I be asking you anything else?

- These questions (and others) will help to "break the ice" and initiate the communication that is essential to successful elicitation. But a question-and-answer meeting format is not an approach that has been overwhelmingly successful.
- In fact, the Q&A session should be used for the first encounter only and then replaced by a requirements elicitation format that combines elements of problem solving, negotiation, and specification. An approach of this type is presented in Eliciting Requirements.

Eliciting Requirements:

- Requirements elicitation (also called requirements gathering) combines elements of problem solving, elaboration, negotiation, and specification. In order to encourage a collaborative, team-oriented approach to requirements gathering, stakeholders work together to identify the problem, propose elements of the solution, negotiate different approaches and specify a preliminary set of solution requirements.

Collaborative Requirements Gathering:

Usage Scenarios:

- As requirements are gathered, an overall vision of system functions and features begins to materialize. However, it is difficult to move into more technical software engineering activities until you understand how these functions and features will be used by different classes of end users.
- To accomplish this, developers and users can create a set of scenarios that identify a thread of usage for the system to be constructed. The scenarios, often called use cases , provide a description of how the system will be used.

Elicitation Work Products:

- The work products produced as a consequence of requirements elicitation will vary depending on the size of the system or product to be built. For most systems, the work products include

    • A statement of need and feasibility

    • A bounded statement of scope for the system or product.

    • A list of customers, users, and other stakeholders who participated in requirements elicitation.

    • A description of the system's technical environment.

    • A list of requirements (preferably organized by function) and the domain constraints that apply to each.

    • A set of usage scenarios that provide insight into the use of the system or product under different operating conditions.

    • Any prototypes developed to better define requirements.

- Each of these work products is reviewed by all people who have participated in requirements elicitation

Developing use cases:

When developing use cases you should start with a functional partition—a listing of the major functional categories of the application. This will help identify what areas need to be focused on.

Step 1: Identify who is going to be using the system directly. These are the Actors.

The main component of use case development is actors. An actor is a specific role played by a system user and represents a category of users that demonstrates similar behaviors when using the system. The actors may be people or computer systems. A primary actor is one having a goal requiring the assistance of the system. A secondary actor is one from which the system needs assistance to satisfy its goal. One of the actors is designated as the system under discussion. A person can play several roles and thereby represent several actors, such as computer-system operator or end user.

Step 2: Pick one of those Actors.

To identify a target system's use case, we identify the system actors. A good starting point is to check the system design and identify who it is supposed to help.

Step 3: Define what that Actor wants to do with the system. Each of these things that the actor wants to do with the system become a Use Case.

The things that the actors want to do with the system become goals. The goal is the end outcome of the actions of the user. There are two types of goals. The first type is a rigid goal. This goal must be completely satisfied and describes a target system's minimum requirement. The second type of goal is a soft goal. This usually describes a desired property for a target system and does not need to be completely satisfied. To identify use cases, we can read the requirement specification from an actor's perspective and carry on discussions with those users who will function as actors. By defining everything that every actor will be able to do in interaction with the system, the complete functionality of the system is defined.

Step 4: For each of those Use Cases decide on the most usual course when that Actor is using the system. What normally happens.

A use case has one basic course and several alternative courses. The basic course is the simplest course, the one in which a request is delivered without any difficulty. (12) There may be alternative courses that describe variants of the basic course and the errors that can occur. These are documented as extensions to the use case.

Step 5: Describe that basic course in the description for the use case.

The use scenario is written from the user's perspective in view in easy to understand language. This step is very similar to documenting a process flow. The steps necessary to achieve the identified goal are written out.

Step 6: Once you're happy with the basic course now consider the alternatives and add those as extending use cases.
The extensions are written in the same manner as the original use case but they provide alternatives to the simplest path.

When analyzing and describing use cases in detail it is not unusual to uncover points that are not clear in the requirement specification. Vague requirements thus become obvious at an early stage and can be corrected before the Design Phase. Once the use cases are complete it is important to discuss with the customer and make any necessary changes. It is important to have user buy in before proceeding to the next step in the process because these use cases will become the foundation of your user requirements and design specifications.

Building The Requirements Model:

- The intent of the analysis model is to provide a description of the required informational, functional, and behavioral domains for a computer-based system.
- The model changes dynamically as you learn more about the system to be built, and other stakeholders understand more about what they really require. For that reason, the analysis model is a snapshot of requirements at any given time. You should expect it to change.
- As the requirements model evolves, certain elements will become relatively stable, providing a solid foundation for the design tasks that follow.
- However, other elements of the model may be more volatile, indicating that stakeholders do not yet fully understand requirements for the system.

Elements Of the Requirement model:

requirements for a computer-based system can be seen in many different ways. Some software people argue that it's worth using a number of different modes of representation while others believe that it's best to select one mode of representation.

The specific elements of the requirements model are dedicated to the analysis modeling method that is to be used.

- **Scenario-based elements :**
  Using a scenario-based approach, system is described from user's point of view. **For example**, basic use cases and their corresponding use-case diagrams evolve into more elaborate template-based use cases. Figure 1(a) depicts a UML activity diagram for eliciting requirements and representing them using use cases. There are three levels of elaboration.
- **Class-based elements :**
  A collection of things that have similar attributes and common behaviors i.e., objects are categorized into classes. **For example**, a UML case diagram can be used to depict a Sensor class for the SafeHome security function. Note that diagram lists attributes of sensors and operations that can be applied to modify these attributes. In addition to class diagrams, other analysis modeling elements depict manner in which classes collaborate with one another and relationships and interactions between classes
- **Behavioral elements :**
  Effect of behavior of computer-based system can be seen on design that is chosen and implementation approach that is applied. Modeling elements that depict behavior must be provided by requirements model.

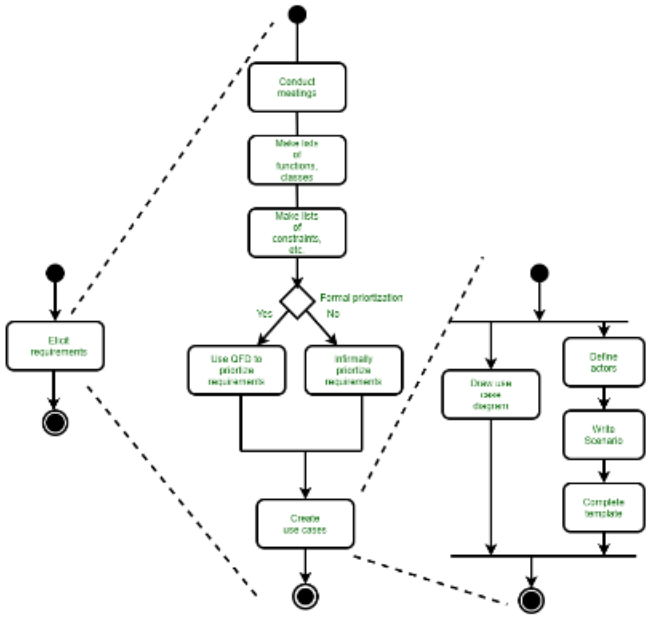
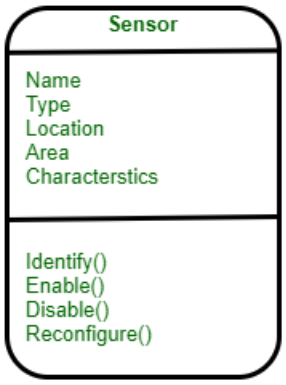


Figure 1(a):UML activity diagrams for eliciting requirements



Class diagram for sensor

Method for representing behavior of a system by depicting its states and events that cause system to change state is state diagram. A state is an externally observable mode of behavior. In addition, state diagram indicates actions taken as a consequence of a particular event.

To illustrate use of a state diagram, consider software embedded within safeHome control panel that is responsible for reading user input. A simplified UML state diagram is shown in figure 2.

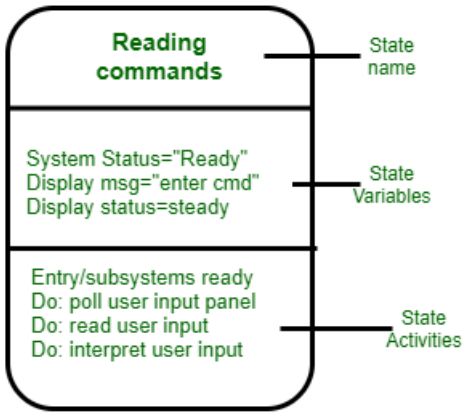


Figure 2: UML state diagram notation

- **Flow-oriented elements :**
  As it flows through a computer-based system information is transformed. System accepts input, applies functions to transform it, and produces output in a various forms. Input may be a control signal transmitted by a transducer, a series of numbers typed by human operator, a packet of information transmitted on a network link, or a voluminous data file retrieved from secondary storage. Transform may compromise a single logical comparison, a complex numerical algorithm, or a rule-inference approach of an expert system. Output produce a

200-page report or may light a single LED. In effect, we can create a flow model for any computer-based system, regardless of size and complexity.

Negotiating Requirements:

- In an ideal requirements engineering context, the inception, elicitation, and elaboration tasks determine customer requirements in sufficient detail to proceed to subsequent software engineering activities. Unfortunately, this rarely happens.
- In reality, you may have to enter into a negotiation with one or more stakeholders. In most cases, stakeholders are asked to balance functionality, performance, and other product or system characteristics against cost and time-to-market.
- The intent of this negotiation is to develop a project plan that meets stakeholder needs while at the same time reflecting the real-world constraints (e.g., time, people, budget) that have been placed on the software team.
- The best negotiations strive for a "win-win" result.That is, stakeholders win by getting the system or product that satisfies the majority of their needs and you (as a member of the software team) win by working to realistic and achievable budgets and deadlines. Boehm defines a set of negotiation activities at the beginning of each software process iteration.
- Rather than a single customer communication activity, the following activities are defined:

  1. Identification of the system or subsystem's key stakeholders.

  2. Determination of the stakeholders' "win conditions."

  3. Negotiation of the stakeholders' win conditions to reconcile them into a set of win-win conditions for all concerned (including the software team).

- Successful completion of these initial steps achieves a win-win result, which becomes the key criterion for proceeding to subsequent software engineering activities

Validating Requirements:

- As each element of the requirements model is created, it is examined for inconsistency, omissions, and ambiguity. The requirements represented by the model are prioritized by the stakeholders and grouped within requirements packages that will be implemented as software increments
- A review of the requirements model addresses the following questions:

  - Is each requirement consistent with the overall objectives for the system/product?
  - Have all requirements been specified at the proper level of abstraction? That is, do some requirements provide a level of technical detail that is inappropriate at this stage?
  - Is the requirement really necessary or does it represent an add-on feature that may not be essential to the objective of the system?
  - Is each requirement bounded and unambiguous?

- Does each requirement have attribution? That is, is a source (generally, a specific individual) noted for each requirement?
- Do any requirements conflict with other requirements?
- Is each requirement achievable in the technical environment that will house the system or product?
- Is each requirement testable, once implemented?
- Does the requirements model properly reflect the information, function, and behavior of the system to be built?
- Has the requirements model been "partitioned" in a way that exposes progressively more detailed information about the system?
- Have requirements patterns been used to simplify the requirements model? Have all patterns been properly validated? Are all patterns consistent with customer requirements?

- These and other questions should be asked and answered to ensure that the requirements model is an accurate reflection of stakeholder needs and that it provides a solid foundation for design